

ICLR 2024 | A Schrödinger Bridge You Can Solve on a Laptop

LightSB collapses entropic optimal transport into a single Gaussian-mixture objective — minutes on a CPU, no adversarial training, with a universal-approximation guarantee.

A Schrödinger Bridge asks a deceptively simple question: given two probability distributions, what is the most natural stochastic path that carries one into the other? "Most natural" here means the diffusion process that stays as close as possible to a reference Wiener process — plain Brownian motion. This object is the dynamic form of Entropic Optimal Transport (EOT), and it turns up all over applied machine learning, from tracking how single cells differentiate over time, to unpaired image translation, to generative modeling.

The catch is that actually computing a Schrödinger Bridge has become an exercise in heavy machinery. Most modern solvers stack several neural networks, lean on adversarial min-max objectives or iterative proportional fitting (IPF), and simulate the full stochastic process at every training step. They demand careful network design, run for hours on a GPU, and are sensitive to a long list of hyperparameters. The field has powerful solvers, but it lacks a simple, principled default — the Schrödinger Bridge equivalent of k-means for clustering or Sinkhorn for discrete optimal transport, the method you reach for first before anything fancier.

The paper Light Schrödinger Bridge (LightSB), from researchers at Skoltech and AIRI, sets out to be exactly that baseline. Its move is to parameterize the Schrödinger potential as a Gaussian mixture and treat the logarithm of that potential as an energy function. Under this choice the whole problem collapses into a single, non-minimax optimization objective whose key pieces — the transport plan and the drift of the process — have closed-form expressions. The result solves Schrödinger Bridges in moderate dimensions in minutes on a CPU, needs almost no hyperparameter tuning, and is provably a universal approximator of Schrödinger Bridges.



Figure 1. Unpaired male-to-female face translation by LightSB in the 512-dimensional ALAE latent space of 1024x1024 FFHQ images. Each row starts from one real male photo (left); because the bridge is stochastic, a single input maps to a spread of plausible outputs. Converges in under one minute on 4 CPU cores.

Paper: <https://arxiv.org/abs/2310.01174>

Code: <https://github.com/ngushchin/LightSB>

The problem: heavy solvers, no baseline

Formally, the Schrödinger Bridge between a source distribution p_0 and a target p_1 is the diffusion process, pinned to those two marginals, that minimizes its Kullback-Leibler divergence from a Wiener prior. A noise level ε controls how much the process is allowed to wander: small ε keeps trajectories nearly straight and deterministic, large ε lets them diffuse widely. Solving the bridge yields both a transport plan $\pi^*(x_1 | x_0)$ — where a given starting point is likely to end up — and the full time-dependent process connecting the two ends.

In most real applications, though, only the endpoints matter. A biologist mapping cells from an early day to a later one cares about the conditional plan $\pi^*(x_1 | x_0)$ — where a cell is likely to end up — not the exact shape of every intermediate trajectory. That mismatch, heavy neural machinery for a comparatively modest goal, is the gap LightSB is built to close. It helps to picture the target in two dimensions first.

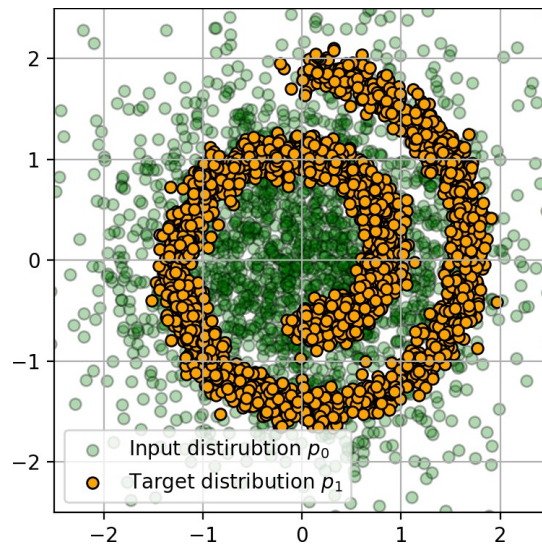


Figure 2. A two-dimensional toy problem: LightSB is asked to bridge an input Gaussian blob (green, the source distribution p_0) to a Swiss-roll ring (orange, the target distribution p_1). Because the solver is lightweight, it is cheap to visualize how the noise level ε reshapes the learned process on a task like this — from near-straight transport at small ε to visibly volatile trajectories as ε grows.

How LightSB works

LightSB starts from the natural goal: directly minimize $KL(\pi^* || \pi_\theta)$, the divergence between the true EOT plan and a parameterized plan π_θ . The obstacle is obvious — we do not know the true plan π^* . The key observation is that we do not have to. The optimal plan is known to factor as $\pi_\theta(x_1 | x_0)$ proportional to $\exp(\langle x_0, x_1 \rangle / \varepsilon) \cdot v_\theta(x_1)$, so all that is left to learn is the adjusted Schrödinger potential v_θ .

LightSB parameterizes that potential as an unnormalized Gaussian mixture, $v_\theta(x_1) = \sum_k \alpha_k \cdot N(x_1 | r_k, \varepsilon S_k)$. This single choice makes two otherwise-intractable quantities available in closed form: the conditional distributions of the plan, $\pi_\theta(x_1 | x_0)$, and the normalization constant $c_\theta(x_0)$. With those in hand, the KL objective rewrites as a simple difference of two expectations — the log-normalization averaged over the source and the log-potential averaged over the target — which can be estimated by Monte Carlo from samples and then minimized

with ordinary stochastic gradient descent (SGD), the same workhorse optimizer used to train any standard neural network.

$$L(\theta) = E_{\{x_0 \sim p_0\}}[\log c_{\theta}(x_0)] - E_{\{x_1 \sim p_1\}}[\log v_{\theta}(x_1)]$$

There is no min-max game, no iterative proportional fitting, and no simulation of trajectories during training — the three ingredients that make earlier solvers slow and fragile. And because the same learned potential yields a closed-form drift, the full bridge process can be reconstructed after the fact whenever the intermediate dynamics are actually needed. The authors also prove that this Gaussian-mixture solver is a universal approximator of Schrödinger Bridges — they note it is the first such result — and show its generalization error vanishes at the standard parametric rate as the sample size grows.

Results: accuracy and speed

The first test is a benchmark where the ground-truth plan is actually known (Gushchin et al., 2023b), spanning dimensions $D = 2$ to 128 and noise levels $\varepsilon = 0.1, 1, 10$. Here LightSB is not merely competitive — it wins by a wide margin. Its conditional Bures-Wasserstein error (cBW-squared-UP) lands between roughly 0.03% and 0.62%, while the best prior solver ranges from about 1.04% up to 18.05%. When the target aligns with LightSB’s Gaussian-mixture inductive bias, the gap is largest.

The more striking result is speed. On the MSCI single-cell dataset — cells from four human donors across several days, projected by PCA to 50, 100, and 1000 dimensions — LightSB matches strong GPU-based solvers in energy distance while training on just 4 CPU cores in 65 to 146 seconds, versus roughly 8 to 71 minutes on a V100 GPU for the continuous baselines. The hardest setting makes the contrast concrete: an energy distance of 1.27 on CPU against 1.32 for a minimax GPU solver, at a fraction of the wall-clock cost.

Table 1. MSCI single-cell task at DIM = 1000: LightSB matches a strong minimax GPU solver on energy distance while training on CPU in a fraction of the wall-clock time (numbers from the paper).

Solver	Energy distance (lower is better)	Hardware	Training time
LightSB (this paper)	1.27	4 CPU cores	146 s
Minimax solver	1.32	V100 GPU	71 min

The same lightness carries over to a much higher-dimensional problem: unpaired face translation in the 512-dimensional latent space of a pretrained ALAE autoencoder. LightSB performs male-to-female and adult-to-child translation there, again converging in under a minute on 4 CPU cores. Because the plan is stochastic, a single input face yields a spread of plausible outputs rather than one fixed answer, which is exactly what a well-behaved stochastic transport plan should do.



Figure 3. Male-to-female translation across two different source faces (left of each pair) and the samples LightSB produces (right). The translations change the target attribute while preserving identity-adjacent details — pose, lighting, expression — all learned in the ALAE latent space without paired training data.

What ϵ buys you

The noise level ϵ is the one knob that genuinely changes the character of the solution, and the 2D Gaussian-to-Swiss-roll study makes its role visible. Sweeping ϵ across small to larger values (the paper uses $2 \cdot 10^{-3}$, 10^{-2} , 10^{-1}), small ϵ produces nearly straight, almost deterministic trajectories that map each source point to a tight endpoint; as ϵ grows, trajectories become more volatile and the conditional distributions $\pi_{\theta}(x_1 | x_0)$ spread out. This is exactly the behavior the theory predicts — ϵ is the dial for how stochastic the bridge is — and it means a user can pick ϵ to match how much diversity they want in the output, rather than tuning it just to appease a finicky optimizer.

The takeaway

The lasting message of LightSB is that a Schrödinger Bridge solver does not have to be heavy. By writing the transport potential as a Gaussian mixture and minimizing one straightforward objective, LightSB solves entropic optimal transport and Schrödinger Bridges in minutes on a CPU — no adversarial training, no process simulation, and little tuning — while matching or beating much heavier GPU-based solvers and carrying a universal-approximation guarantee.

The honest boundary is scope. LightSB targets moderate-dimensional distributions, and its image results live in a pretrained autoencoder's latent space rather than in raw pixel space, so it is a transport solver rather than a from-scratch image generator. Within that scope, though, it does something the field had been missing: it offers a simple, fast, reliable default — the baseline you can run first, on the hardware you already have, before deciding whether a heavier solver is worth it.